Spring MVC: The Request Flow

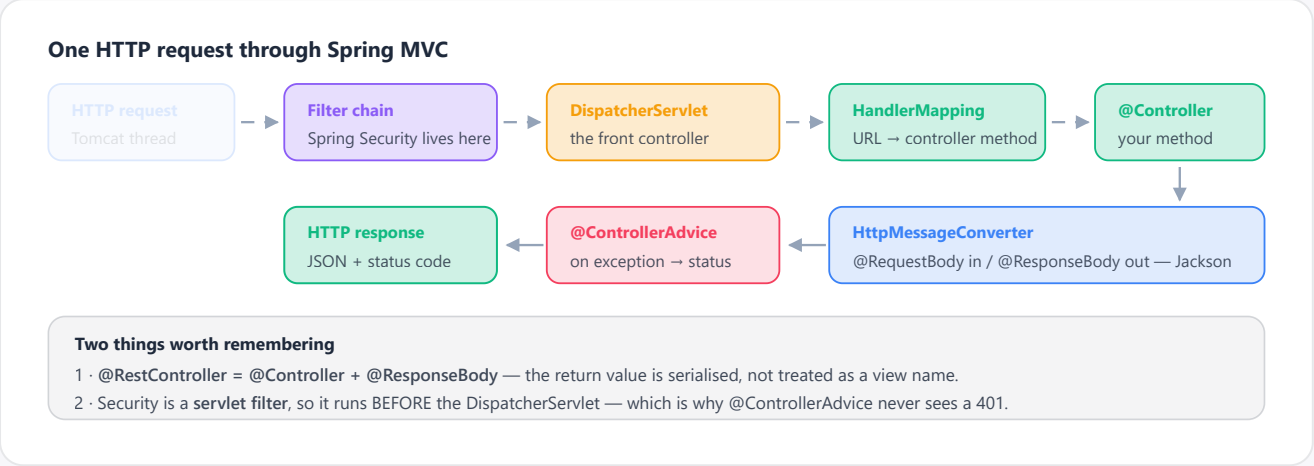
7 min read · Updated 28 Jul 2026 · Topic 55 of 65

Why it matters

Tracing one HTTP request end to end explains a surprising number of otherwise-baffling behaviours: why a security failure never reaches your `@ControllerAdvice`, why a validation error produces a 400 you did not write, and why returning a `String` from `@Controller` and `@RestController` does completely different things.

The concept, simply

Spring MVC is a **front controller**: every request enters through one servlet, the `DispatcherServlet`, which orchestrates the rest.



The filter chain sits BEFORE the DispatcherServlet. That is why Spring Security — which is a filter — rejects a request without any of your exception handling ever running.

How it works

The path: servlet **filters** run first (security, CORS, logging, request wrapping). Then the `DispatcherServlet` consults **handler mappings** to find the controller method for the URL and verb. **Argument resolvers** then bind the request to the method's parameters —

a converter. If anything throws, `@ControllerAdvice` handlers get a chance to translate it.

`@RestController` is `@Controller` + `@ResponseBody`. Without `@ResponseBody`, a returned `String` is interpreted as a *view name* and Spring tries to resolve a template — the source of a great many confusing 404s in code converted from MVC to REST.

Code examples

JAVA

Copy

```java
@RestController
@RequestMapping("/api/orders")
public class OrderController {
    private final OrderService service;
    public OrderController(OrderService service) { this.service = service;

    @GetMapping("/{id}")
    public OrderResponse get(@PathVariable String id) {
        return service.find(id)                    // Optional (Chapter 7
                      .map(OrderResponse::from)
                      .orElseThrow(() -> new OrderNotFoundException(id));
    }

    @GetMapping
    public Page<OrderResponse> list(
            @RequestParam(defaultValue = "0") int page,
            @RequestParam(required = false) String status,
            Pageable pageable) { … }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public OrderResponse create(@Valid @RequestBody CreateOrderRequest req
        return OrderResponse.from(service.create(req));
    }
}
// @Valid triggers Bean Validation on the body. A violation throws
```

Centralised error handling:

JAVA

Copy

```java
@RestControllerAdvice
public class ApiExceptionHandler {

    @ExceptionHandler(OrderNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public ErrorBody notFound(OrderNotFoundException e) {
        return new ErrorBody("ORDER_NOT_FOUND", e.orderId());    // Chapter
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorBody> invalid(MethodArgumentNotValidExcepti
        var fields = e.getBindingResult().getFieldErrors().stream()
                .collect(toMap(FieldError::getField, FieldError::getDefaul
        return ResponseEntity.badRequest().body(new ErrorBody("VALIDATION"
    }

    @ExceptionHandler(Exception.class)          // last-resort boundary
    public ResponseEntity<ErrorBody> unexpected(Exception e) {
        log.error("unhandled", e);              // ✓ log with the exceptic
        return ResponseEntity.internalServerError()
                             .body(new ErrorBody("INTERNAL", null));
        // ✓ never leak the exception message to the client — it can expos
        //   SQL, file paths or internal hostnames.
    }
}

// ⚠ This never sees a Spring Security 401/403 raised in the filter chain,
//   because filters run before the DispatcherServlet. Those are configure
//   with an AuthenticationEntryPoint / AccessDeniedHandler instead.
```

Modern Boot 3 note worth knowing:

JAVA

Copy

```
// and ResponseEntityExceptionHandler produces ProblemDetail objects.
@ExceptionHandler(OrderNotFoundException.class)
public ProblemDetail handle(OrderNotFoundException e) {
    var pd = ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, e.getM
    pd.setType(URI.create("https://api.example.com/errors/order-not-found"
    pd.setProperty("orderId", e.orderId());
    return pd;
}
```

Under the hood

The thread model matters for everything in Chapter 8. Traditional Spring MVC is one thread per request from the servlet container's pool, and that thread is occupied for the whole request — including while it waits on a database or an HTTP call. This is why a slow dependency exhausts the pool and the whole application stops responding, and why pool sizing is a capacity decision rather than a detail. On Java 21, Boot can run those requests on virtual threads (`spring.threads.virtual.enabled=true`), which removes the constraint entirely — the modern answer to "how do you handle a slow downstream?".

Filters versus interceptors is a standard follow-up. Filters are Servlet API, run outside Spring MVC, and see the raw request and response — right for security, CORS and logging. Interceptors are Spring MVC, run inside the `DispatcherServlet`, and know which handler method was selected — right for anything needing that context, like per-endpoint auditing. If you need the handler, use an interceptor; if you need to sit in front of everything, use a filter.

Never let exception details reach the client. A default error page or a leaked `getMessage()` can expose SQL fragments, file paths, library versions and internal hostnames — all useful to an attacker. Log the detail server-side with the exception object, return a stable error code and a generic message.

Common mistakes

- **Using `@Controller` for a REST API.** Return values become view names.

- **Expecting `@ControllerAdvice` to catch security errors.** Filters run before the dispatcher.
- **Returning entities directly.** Leaks the schema and lazy associations — use a DTO.
- **Forgetting `@Valid`.** The constraints on the DTO are then decorative.
- **Leaking exception messages.** Log detail, return a code.
- **Business logic in controllers.** Keep them thin — bind, delegate, map.
- **Blocking calls without considering the thread pool.** One slow dependency stalls everything.

Interview lens

Q: "Walk me through a request in Spring MVC."
A: It passes the servlet filter chain first — that is where Spring Security sits. Then the `DispatcherServlet` takes over: handler mapping selects the controller method from URL and verb, argument resolvers bind path variables, query parameters and the body via an `HttpMessageConverter`, the method runs, and the return value is serialised back through a converter. If it throws, `@ControllerAdvice` translates the exception into a status and body.

Q: "`@Controller` vs `@RestController`?"
A: `@RestController` is `@Controller` plus `@ResponseBody` on every method, so return values are serialised into the response body. With plain `@Controller`, a returned `String` is treated as a view name and Spring tries to resolve a template — which is the cause of a lot of unexplained 404s when someone converts an MVC controller to REST.

Q: "Why doesn't my `@ControllerAdvice` catch authentication failures?"
A: Because Spring Security is a servlet filter and filters run before the `DispatcherServlet`. The request is rejected before any controller or exception handler is involved. Those responses are customised with an

`AuthenticationEntryPoint` for 401s and an `AccessDeniedHandler` for 403s, in the security configuration rather than in MVC.

Q: "Filter or interceptor?"

A: A filter when I need to sit in front of everything and work with the raw request and response — security, CORS, request logging, wrapping the body. An interceptor when I need to know which handler method was selected, since it runs inside the `DispatcherServlet` and has that context — per-endpoint auditing or metrics, for example.

Q: "How do you handle validation?"

A: Bean Validation constraints on the request DTO plus `@Valid` on the parameter. A violation throws `MethodArgumentNotValidException`, which Boot already maps to a 400 with a structured body, so I usually only add a handler to reshape the response into our error format. Boot 3 makes that easier with RFC 7807 problem details.

Q: "What is the thread model, and what happens with a slow dependency?"

A: One thread per request from the container pool, held for the entire request including all blocking I/O. So a slow downstream call ties up threads until the pool is exhausted and the whole application stops accepting work. Traditionally you would bound it with timeouts and a circuit breaker. On Java 21 with `spring.threads.virtual.enabled=true`, requests run on virtual threads and the constraint largely disappears.

Cheat-card

- Path: **filters → DispatcherServlet → handler mapping → argument resolvers → controller → message converter**.
- **Security is a filter** → runs before MVC → `@ControllerAdvice` never sees 401/403.
- `@RestController` = `@Controller` + `@ResponseBody`.

- **Return DTOs, never entities.** Never leak exception messages to clients.
- Filter = raw request, no handler context. Interceptor = inside MVC, knows the handler.
- **One thread per request, held across blocking I/O** → Java 21: `spring.threads.virtual.enabled=true`.
- Boot 3 error format: **RFC 7807** `ProblemDetail`.

QUICK CHECK — no marks, just for you.

1. Why does @ControllerAdvice never see a Spring Security 401?

- ◯ Because 401 responses are generated by the servlet container
- ◯ Because security exceptions are Errors, not Exceptions
- ◯ Because @ControllerAdvice only handles exceptions from the service layer
- ◯ Because Spring Security is a servlet FILTER, and filters run before the DispatcherServlet — the request is rejected before any controller or exception handler is involved

Check answer

2. What is the difference between @Controller and @RestController?

- ◯ @RestController is @Controller plus @ResponseBody, so return values are serialised into the body rather than treated as view names
- ◯ @RestController only supports GET requests
- ◯ @Controller is deprecated
- ◯ @RestController automatically adds validation

3. When would you use a servlet Filter rather than a Spring interceptor?

- ◯ Filters are deprecated in Spring Boot
- ◯ When you need to sit in front of everything and work with the raw request and response — security, CORS, request logging, body wrapping
- ◯ There is no practical difference
- ◯ When you need to know which controller method was selected

Check answer

‹ Previous
Spring Boot Auto-Configuration

Next ›
Entities, Mapping and Repositories

LEARN MORE

ARTICLES & DOCS

Spring MVC — Spring Framework reference
docs.spring.io

DispatcherServlet — Spring Framework reference
docs.spring.io

VIDEOS



Spring Boot : IOC & Dependency Injection | Code your 1st Spring Boot project | Spring core
Selenium Express

Hand-picked references for this lesson.



Java Interview Kit